

FLP 不可能性结果

任益驰 221900329

2026 年 6 月 28 日

目录

1	引言	2
1.1	主要问题	2
1.2	相关研究	2
1.3	核心结论	2
2	定义	3
2.1	术语	3
2.2	模型	4
3	证明	5
3.1	LEMMA 1	5
3.2	LEMMA 2	5
3.3	LEMMA 3	6
3.4	THEOREM 1	7
3.5	THEOREM 2	8
4	结论	9

1 引言

1.1 主要问题

分布式共识是分布式计算理论中最核心的问题之一。基本形式为：一组进程，每个进程持有一个输入值，它们需要通过彼此通信，最终全体一致地对一个输出值达成协议，且该输出值必须是某个进程的输入值。

形式化来说，一个共识协议需满足以下三条性质：

1. **一致性 (Agreement)**：所有节点达成一致的决议。
2. **有效性 (Validity)**：输出值必须是某个进程的输入值。也就是如果所有节点中的数据只有 0 和 1 两种，那么最后达成一致的决议一定是 0 和 1 中的一种，不会出现其他值。
3. **终止性 (Termination)**：所有正常运行的节点最终都能够做出决议。

共识问题是许多分布式系统的基础构件，一种广为人知的形式是事务提交问题，常见于分布式数据库系统。在该场景中，多个数据管理器进程需协同决定是否将某事务的结果永久写入数据库或全部撤销，共识问题是保证数据一致性的核心机制。因此，共识的可解性是一个既有理论深度又有工程意义的重要问题。

1.2 相关研究

在本文的工作之前，分布式系统领域已经对共识问题有了一定的认识。研究者们已经证明：在同步系统中，即使存在进程崩溃故障，共识问题也是可解的（如 Pease-Shostak-Lamport 的拜占庭将军问题）。然而，同步假设——即消息传递有已知的上界、进程步调以轮次同步推进——在现实系统中往往无法严格保证。

一个自然的问题是：如果我们放松同步假设，考虑异步系统（其中消息延迟无上界、进程之间没有公共时钟），共识问题是否仍然可解？

1.3 核心结论

Fischer、Lynch 和 Paterson 在论文《Impossibility of Distributed Consensus with One Faulty Process》中给出了一个令人震惊的结论：在消息系统可靠（没有丢包，消息不会被重复传递）的异步系统中，即使只允许一个进程崩溃故障，也无法保证共识问题的可解性。换句话说，在异步系统中，任何共识协议都无法同时满足一致性、有效性和终止性三条性质。

该结果被称为 FLP 不可能性结果，它揭示了分布式计算理论中的一个基本限制，并对后续的研究产生了深远影响。

2 定义

2.1 术语

Process: 在该论文中可以把它理解为分布式系统中的一个节点。每一个 process p 有一个 input register x_p , 可以理解为输入值, 只有 0 和 1 两种。此外 process p 还有一个 output register y_p , 可以理解为输出值, 有 b , 0 和 1 三种。即 $x_p \in \{0, 1\}, y_p \in \{b, 0, 1\}$ 。

Message system: 在该论文中可以把它理解为分布式系统中的一个通信网络, 连接了整个集群中的所有 process。整个系统中所有的 process 均共用这一个 message system, process 通过这个进行相互之间的通讯。

Asynchronous system: 异步系统。在该论文中对异步系统的核心假设是: 没有任何关于时间的信息可用。具体来说: 1) 没有任何关于消息传递延迟的假设。消息可以在任意时间被传递, 甚至可能永远不会被传递。2) 没有任何关于进程执行速度的假设。进程可以以任意速度执行, 甚至可能永远不会执行。3) 没有同步时钟。进程之间没有任何同步时钟, 无法通过时间来协调彼此的行为, 无法使用超时机制。4) 没有任何关于进程崩溃的假设。进程可能会崩溃, 但无法通过时间来判断一个进程是否已经崩溃, 无法区分“进程已崩溃”和“进程只是跑得非常慢”。

Internal state: 内部状态。文中将 Internal state 定义为一个元组, 即 $\text{internal state} = (\text{input register}, \text{output register}, \text{program counter}, \text{internal storage})$ 。

Configuration: 配置。一个 Configuration 包含了整个集群中所有 process 的 internal state 外加 message system。

Step / Event: 事件。事件 $e = (p, m)$ 表示消息 m 被投递给进程 p 。将事件 e 应用于配置 C 得到新配置 $e(C)$, 其中进程 p 根据其内部状态和消息 m 执行一步计算, 更新内部状态并可能发送新消息。

Schedule: 调度。调度是一个事件的序列 $\sigma = e_1 e_2 \dots$ (可能无限)。将有限调度 σ 应用于配置 C 得到新配置 $\sigma(C) = e_n(\dots e_2(e_1(C))\dots)$ 。

Run: 运行。运行是由配置构成的 (可能无限) 序列 $C_0, C_1, C_2, \dots$, 使得存在调度 $\sigma = e_1 e_2 \dots$ 满足 $C_i = e_i(C_{i-1})$ 。

Admissible run: 可接受的运行。一个运行是可接受的, 如果它满足以下两个条件: (1) 至多有一个进程崩溃; (2) 所有发往非故障进程的消息最终都能被收到。

Deciding run: 决策运行。一个运行是决策的, 如果至少有一个进程在该运行中做出了决策。

Partially correct: 部分正确。一个共识协议是部分正确的, 需要满足两个条件: (1) 不存在任何一个可达到的 configuration 同时包含两种不同的决议值 (即不会出现矛盾)。(2) 对于每种可能的决议值 (0 和 1), 都存在某个可达到的 configuration 能够做出该决议 (即两种值都有可能被选中, 排除掉那种永远只输出

0 的平凡解)。简单来说, 部分正确只保证”不会错”, 但不保证”一定能出结果”。

Totally correct in spite of one fault: 单故障下完全正确。一个共识协议如果在部分正确的基础上, 还满足”每一个 admissible run 都是 deciding run”, 则称该协议是单故障下完全正确的。其中 admissible run 是指”至多一个进程故障, 且所有发往非故障进程的消息最终都能被收到”的运行, deciding run 是指”某个进程在该运行中做出了决策”的运行。简单来说, 完全正确比部分正确多了一个”活性”要求——只要故障不超过一个、消息最终能送达, 协议就必须在有限时间内给出结果。

Bivalent: 二值 (或”未锁定”)。对于一个 configuration C , 如果从 C 出发可以到达的 configuration 中, 既存在决议值为 0 的 configuration, 也存在决议值为 1 的 configuration, 则称 C 是 bivalent 的。也就是说, 系统从 C 出发, 既有可能最终决定为 0, 也有可能最终决定为 1, 尚未锁定方向。

Univalent: 单值 (或”已锁定”)。与 bivalent 相对, 如果从 C 出发可以到达的 configuration 中, 所有可能路径都通向同一种决议值, 则称 C 是 univalent 的。具体来说, 若只通向 0 则称 0-valent, 只通向 1 则称 1-valent。系统一旦进入 univalent 状态, 命运的走向就已经确定了, 尽管可能还没有进程实际做出决策。

2.2 模型

在这一部分, 我来简单介绍论文建立的模型, 其中包括:

1. 宽松的分布式环境

- (a) 不考虑 Byzantine 故障, 即数据在传输过程不会被篡改。
- (b) 假设消息系统可靠, 不会丢失数据包, 也不会重复传输数据包。但是不保证数据传输的先后顺序, 而且有时候会返回空 ($\emptyset$)。
- (c) 存在至多一个 process 会在某一个不确定的时间 dead。
- (d) 为了简化, 论文假设所有 process 的 input register $x_p \in \{0, 1\}$, 所有未崩溃的 process 的 output register 在做出决策后取值为 $\{0, 1\}$ (初始值为 b , 表示未决定)。进程的转移函数在进入决策状态后不能再改变输出寄存器中的值, 即 output register 是 **write-once** (只写一次) 的。而且对于正常的共识算法, 肯定是要保证所有的非故障 process 必须选择相同值, 但是论文在这里放宽了条件, 只要求至少存在某个进程最终做出决策。
- (e) 前面提到 message system 可能返回空 ($\emptyset$), 但是论文假设了如果一个 process 不停从 message system 中接受 message, 那它终究能够接收到本该发给它的 message。这样就模拟了网络的延迟, 但是消息一定会送达。

2. 消息系统

Process 之间的通讯是通过消息系统实现的。message 的格式为 (p, m) , p 是 process, m 是 message 的内容。Message system 支持两种操作:

- (a) $\text{send}(p, m)$: 将消息 (p, m) 放入 message system 中。
- (b) $\text{receive}(p)$: 从 message system 中取出一个发往 p 的消息 (p, m) , 如果没有则返回空 $(\emptyset)$ 。

模型的核心特征: 整个模型可以概括为一组确定型自动机 + 一个异步消息系统。其中: (1) 每个 process 是确定型的——给定相同的内部状态和输入消息, 必然产生相同的后续行为。系统的所有非确定性来源于消息系统的调度, 而非进程本身的行为。(2) 消息系统是唯一的非确定性来源——它可以选择何时投递哪条消息、何时返回空 $(\emptyset)$ 。(3) 两个层面通过”事件”交互: 事件 $e = (p, m)$ 表示消息 m 被投递给进程 p , 触发 p 执行一步计算。论文中所有系统状态的演化都通过事件序列 (即调度, schedule) 来描述, 而非通过时间轴。

3 证明

3.1 LEMMA 1

引理内容: 假设从某个配置 C 出发, 分别应用调度 σ_1 和 σ_2 后到达配置 C_1 和 C_2 。如果 σ_1 和 σ_2 中涉及的进程集合互不相交, 那么 σ_2 也可以应用到 C_1 , σ_1 也可以应用到 C_2 , 而且两种顺序最终到达同一个配置 C_3 。

证明思路: 这个引理其实很好理解。两个调度涉及的进程完全不同, 所以它们各自做的事情互不干扰——进程 p 不会影响进程 q 的内部状态, 反之亦然。举个例子, 假设进程 1 和进程 2 各自做自己的事, 那么先做进程 1 的事再做进程 2 的事, 和先做进程 2 的事再做进程 1 的事, 系统最终的状态是一样的。这个性质结合操作系统和数据库课程中的相关知识可以很容易理解。

3.2 LEMMA 2

引理内容: 协议 $\mathcal{P}$ 一定存在一个 bivalent 的初始配置。

证明思路: 这个引理用反证法来证。假设不存在 bivalent 的初始配置, 也就是说所有初始配置要么是 0-valent, 要么是 1-valent。

因为我们要求协议是部分正确的, 所以一定存在某个初始配置是 0-valent, 也一定存在某个初始配置是 1-valent (不然就 trivial 了, 比如全部只输出 0 的方案显然不算共识)。

现在考虑两个初始配置, 如果它们只在某一个进程 p 的初始值上不同, 就称它们是”相邻”的。任意两个初始配置之间都可以通过一串相邻的配置连接起来 (就

是从一个状态经过多次一位上的修改一定可以得到目标状态)。那么在 0-valent 和 1-valent 的初始配置之间, 必然存在一对相邻的配置 C_0 和 C_1 , 其中 C_0 是 0-valent, C_1 是 1-valent, 且它们只在进程 p 的初始值上有区别。

因为协议 $\mathcal{P}$ 是单故障下完全正确的 (定理 1 证明的反证假设), 每个 admissible run 都是 deciding run。从 C_0 出发可以构造一个永远不包含进程 p 事件的 admissible run (异步系统允许无限期不调度 p , p 可视为已故障), 由完全正确性该 run 一定是 deciding 的。设其调度为 σ 。因为 σ 不涉及 p , 所以它也可以应用到 C_1 上。两个运行除了 p 的内部状态外是完全一样的, 因此它们最终会做出相同的决议。

如果这个决议是 1, 那说明从 C_0 既能走到决议 0 (因为 C_0 是 0-valent), 也能走到决议 1 (我们刚构造出来了), 所以 C_0 实际上是 bivalent 的。如果决议是 0, 那么 C_1 是 bivalent 的。不管哪种情况, 都和“不存在 bivalent 初始配置”的假设矛盾。

因此, 一定存在一个 bivalent 的初始配置。这个引理是整个证明的起点——它告诉我们系统一开始是有“选择空间”的。

3.3 LEMMA 3

引理内容: 设 C 是一个 bivalent 的 configuration, $e = (p, m)$ 是一个可以应用到 C 上的事件。令 $\mathcal{D}$ 是从 C 出发、不经过 e 所能到达的所有 configuration 的集合, $\mathcal{E} = e(\mathcal{D}) = \{e(C') \mid C' \in \mathcal{D} \text{ 且 } e \text{ 可应用到 } C'\}$ 。那么 $\mathcal{E}$ 中一定包含一个 bivalent 的 configuration。

直观理解: 这个引理的意思是: 不管你想在系统上施加哪个事件 e , 你都可以通过“绕路”的方式——先走一些别的步骤, 最后再执行 e ——仍然落在一个 bivalent 的 configuration 上。也就是说没有任何一个事件能强迫系统进入 univalent 状态, 你总有办法保持系统的开放性。

证明思路: 还是用反证法。假设 $\mathcal{E}$ 中所有 configuration 都是 univalent 的, 我们来导出矛盾。

因为 C 是 bivalent 的, 从 C 出发既有可能走到决议 0 也有可能走到决议 1, 再根据假设 ($\mathcal{E}$ 中所有 configuration 都是 univalent 的) 可以知道 $\mathcal{E}$ 中既有 0-valent 的 configuration $\mathcal{E}_0$, 也有 1-valent 的 configuration $\mathcal{E}_1$ 。进一步地, 在 $\mathcal{D}$ 中存在两个“邻居”配置 C_0 和 C_1 (即 $C_1 = e'(C_0)$, 只差一个事件), 使得 $\mathcal{E}_0 = e(C_0)$ 、 $\mathcal{E}_1 = e(C_1)$ 。现在分两种情况讨论:

情况 1: e' 涉及另一个进程 q ($q \neq p$)。这时候 Lemma 1 就派上用场了——事件 e 和 e' 涉及不同进程, 所以它们可交换。也就是说从 C_0 出发, 先 e 后 e' 和先 e' 后 e 到达同一个 configuration。但 $e(C_0)$ 是 0-valent, 它的后继应该也是 0-valent; 而 $e'(e(C_0)) = e(e'(C_0)) = e(C_1)$ 却是 1-valent。这就矛盾了。

情况 2: $e' = (p, m')$, 和 e 是同一个进程的事件。Lemma 1 在这里用不了（两个事件涉及同一个进程）。我们需要换一种思路来导出矛盾：

1. 由完全正确性（每个 admissible run 都是 deciding run），从 C_0 出发构造一个 deciding run σ ，其中进程 p 不执行任何步骤（异步系统允许我们无限期地不调度 p ）。设 $\sigma(C_0) = C'_0$ ， C'_0 做出了某个决议值 v 。
2. 同样的 σ 也可以应用到 C_1 上。因为 σ 不含 p 的步骤，而 e' 只涉及 p ，由 Lemma 1， σ 和 e' 可交换。所以 $\sigma(C_1) = \sigma(e'(C_0)) = e'(\sigma(C_0)) = e'(C'_0)$ ，记作 C'_1 。 C'_0 和 C'_1 只在 p 的内部状态上有差别，而 σ 中做决议的进程不是 p ，所以 C'_1 也决议为 v 。
3. 在 C'_0 上应用 $e = (p, m)$ ，得到 $e(C'_0)$ 。因为 σ 和 e 也可交换（同样由 Lemma 1），所以 $e(C'_0) = e(\sigma(C_0)) = \sigma(e(C_0))$ 。而 $e(C_0)$ 是 0-valent，由 valency 的保序性（若配置 C 是 v -valent，则从 C 出发可到达的任何配置也是 v -valent），从它出发经过 σ 到达的 $e(C'_0)$ 也一定是 0-valent。同理， $e(C'_1) = \sigma(e(C_1))$ ，而 $e(C_1)$ 是 1-valent，所以 $e(C'_1)$ 一定是 1-valent。
4. 现在换个角度看 $e(C'_0)$ ： C'_0 已经做出了决议 v ，而且 output register 是 write-once 的（一旦决定就不能改），所以从 C'_0 走一步到 $e(C'_0)$ ，决议值不变，仍为 v 。因此 $e(C'_0)$ 的决议值是 v 。结合上一步可知 $v = 0$ 。
5. 用同样的方式看 $e(C'_1)$ ： C'_1 决议为 v ，write-once 性质保证 $e(C'_1)$ 也决议为 v 。结合上一步可知 $v = 1$ 。

于是得到 $v = 0$ 且 $v = 1$ ，矛盾。矛盾的关键在于， $e(C_0)$ 是 0-valent 而 $e(C_1)$ 是 1-valent 这一事实，通过 σ 和 e 的可交换性传递到了 $e(C'_0)$ 和 $e(C'_1)$ 上，而 C'_0 和 C'_1 的 write-once 性质又把同一个 v 同时赋值 0 和 1。两个方向互相冲突，所以假设不成立。

两种情况都推出了矛盾，所以 $\mathcal{E}$ 中必然存在 bivalent 的 configuration。这个引理保证了我们可以无限期地”推迟”决策。

3.4 THEOREM 1

定理内容：不存在能容忍单个故障进程的完全正确协议。也就是说，不存在一个协议同时满足”部分正确”且”每个 admissible run 都是 deciding run”。直观上来理解就是：在异步系统中，即使只存在一个进程故障，也无法保证可解性，会存在一条路径，使得系统从一个 bivalent 的状态出发，永远无法进入 univalent 状态，也就是永远无法做出决策。

证明思路：用反证法。假设存在这样的协议 $\mathcal{P}$ ，我们构造一个 admissible 但不是 deciding 的运行来推翻它。

构造过程如下：

1. 由 Lemma 2, 存在一个 bivalent 的初始配置 C_0 , 我们从这里出发。
2. 维护一个进程队列, 初始顺序任意。消息缓冲区中的消息按发送时间排序。
3. 每一轮 (称为一个 stage), 取出队首进程 p :
 - (a) 检查 p 的消息缓冲区: 有消息就取最早的那条记为 m , 没有则令 $m = \lambda$ 。
 - (b) 令 $e = (p, m)$ 。根据 Lemma 3, 存在一条路径从当前 bivalent 配置出发, 经过一些步骤 (最后一步是 e), 到达另一个 bivalent 配置。我们就沿着这条路径走。
 - (c) 把 p 移到队尾。
4. 无限重复步骤 3。

这个构造保证了每个进程在队列中无限次出现, 所以每个进程都会执行无限多步, 而且每条消息最终都会被接收。所以这个运行是 admissible 的。

但因为每一步都通过 Lemma 3 保证了停留在 bivalent 状态, 系统永远不会进入 univalent, 也就是说没有任何进程会做出决策。所以这是一个 admissible 但不 deciding 的运行。

这就和“每个 admissible run 都是 deciding run”矛盾了。定理得证。

这个证明主要是基于上面的两个引理。其中 Lemma 2 保证我们一定有一个 bivalent 的起点, 而 Lemma 3 保证我们可以无限期地“绕路”, 永远不进入 univalent 状态。在异步系统中是无法通过超时等手段打破这种拖延过程的 (即使 p 的消息缓冲区一直是空), 因为判断不出一个进程是死了还是执行太过缓慢。

3.5 THEOREM 2

定理内容: 存在一个部分正确的共识协议, 它能在满足以下条件时让所有非故障进程都做出决策: (1) 没有进程在协议执行过程中死亡; (2) 初始时严格多数的进程处于存活状态。

协议的大致思路: 这个协议通过构造一个有向图来解决共识问题。

第一阶段, 每个进程广播自己的编号, 然后等待收到至少 $k-1$ 个其他进程的回复 (连同自己共计 $k = \lceil (n+1)/2 \rceil$ 个进程)。如果进程 p 收到了 q 的回复, 就在图 Γ 中加一条边 $q \rightarrow p$ 。

第二阶段, 每个进程构建 Γ 的传递闭包 Γ^+ 。每个进程把自己的编号、初始值和第一阶段收到的 $k-1$ 个邻居广播给所有人, 然后等待接收来自所有已知祖先的第二阶段消息。通过不断重复这个过程, 每个进程最终都能知道所有指向自己的祖先以及它们的初始值。

最后，每个进程找出 Γ^+ 中的“初始团”——一个没有入边的团。可以证明这样的团有且只有一个（否则可以继续迭代扩大这个团），且大小至少为 k 。因为所有非故障进程都知道这个团中所有成员的初始值，所以它们可以用事先约定好的规则（比如取最小值）做出相同的决策。

为什么这个协议不违反 Theorem 1: Theorem 2 要求“执行过程中没有进程死亡”，而 Theorem 1 要求的是“即使执行过程中有一个进程死亡也能搞定”。二者的条件不同，所以不矛盾。Theorem 2 实际上给 Theorem 1 的结论划了一条边界——如果在整个执行过程中所有进程都活着，那共识还是有可能的。

4 结论

该论文主要是通过反证法的方式，通过构造 bivalent 的初始配置和无限期的绕路，证明了在异步系统中，即使只允许一个进程崩溃故障，也无法保证共识问题的可解性。也就是说，任何共识协议都无法同时满足一致性、有效性和终止性三条性质。